

Tecnologias de Reuso: Componentes e Middleware

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

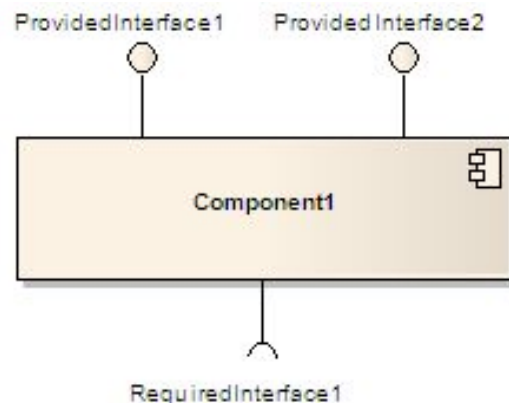
- Introdução ao Reuso de Software
- Componentes de Software
- Middleware
- Atividades Práticas (Componentes e Middleware)
- Usos Reais em Arquiteturas Modernas

Por que tecnologias de reuso?

- Componentes permitem:
 - Reduzir custos
 - Minimizar bugs (componentes já testados)
 - Velocidade de entrega
 - Padronização da arquitetura
- Middleware permite:
 - Reduzir acoplamento entre módulos
 - Adicionar funcionalidades transversais sem modificar código existente
 - Escalabilidade arquitetural

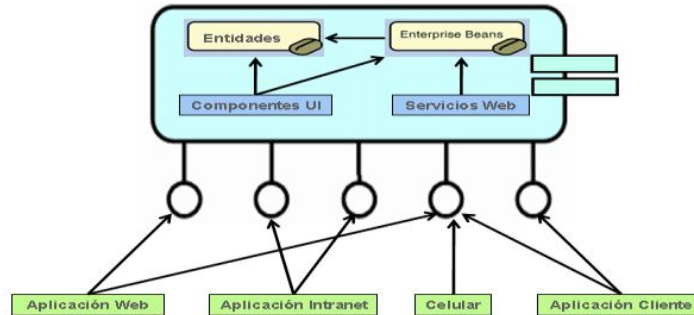
O que é um componente?

- Um componente de software é uma unidade autônoma, reutilizável e substituível, com interface bem definida, desenvolvida para ser integrada a outros sistemas com mínimo acoplamento.
- (Szyperski, 2002 — Engenharia de Componentes)



Uma metáfora simples

- Pense em componentes como peças Lego — qualquer peça encaixa em outra porque segue um padrão de interface.
- A ideia principal é: não é o sistema que precisa se adaptar ao componente; é o componente que deve respeitar um contrato universal.



Propriedades fundamentais

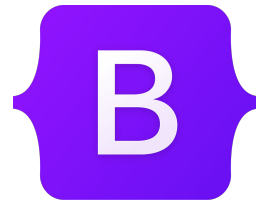
- Encapsulamento
 - Esconde a implementação interna.
 - Permite evoluir sem quebrar consumidores.
 - Fornece apenas o que é necessário via interface.
- Baixo acoplamento
 - Um módulo não depende de detalhes do outro.
 - Reduz impacto de mudanças.
 - Facilita testes unitários e mocks.
- Interface explícita (contrato)
 - Define o que o componente faz, não como.
 - Exemplo: métodos públicos de uma classe, endpoints de API, eventos de um componente front-end

Propriedades fundamentais

- Alta coesão
 - Focado em uma responsabilidade clara.
 - Não tenta resolver “tudo ao mesmo tempo”.
- Independência de plataforma
 - Quando possível, deve funcionar em diferentes:
 - plataformas (Android, Web, iOS),
 - linguagens,
 - ambientes de execução.
- Instalável, configurável e substituível
 - Um componente deve poder ser:
 - instalado (NPM, Maven, pip),
 - configurado (env, parâmetros, JSON),
 - removido ou trocado sem reescrever o sistema.

Componentes: Exemplos de alto impacto no mercado

- Front-end
 - Vuetify (Vue)
 - Bootstrap Components
 - Jetpack Compose (Android)
 - Ex: Button, Card, LazyColumn, Snackbar
- Back-end
 - Spring Boot Starters (spring-boot-starter-web, security, jpa)
 - NestJS Modules (AuthModule, UserModule)
 - Django apps plugáveis
- Componentes de infraestrutura
 - Firebase SDK (Auth, Storage, Firestore)
 - Stripe Payments
 - Auth0 SDK
 - AWS Cognito Client



django



Tipos de Componentes

- Componentes Funcionais
- Componentes de Interface (UI Components)
- Componentes de Domínio
- Componentes Técnicos

Componentes Funcionais

- Implementam funcionalidades completas e reutilizáveis.
 - Autenticação
 - Pagamentos (Stripe)
 - Upload e processamento de arquivos
 - Geração de relatórios PDF
 - Regras de autorização (RBAC)
- Exemplo: `PaymentComponent.process(paymentData)`

Componentes de Interface (UI Components)

- Botões, cards, tooltips, tabelas
 - Seguem um design system (Google Material, iOS Human Interface, Tailwind UI)
 - Reutilizados em dezenas de telas
- Exemplo:
 - `<AppButton label="Salvar" />`

Componentes de Domínio

- Encapsulam lógica de negócio específica.
- Representam conceitos centrais do sistema.
- Geralmente fazem parte do Domain Layer (DDD).
- Exemplo:
 - PagamentoService,
 - AlunoMatriculaComponent,
 - AvaliacaoFisicaDomainModule.

Componentes Técnicos

- Cache
- Logger
- Clients de API

Boas práticas na construção de componentes

- Expor apenas o que precisa ser exposto (interfaces claras)
- Evitar dependências desnecessárias
- Documentar API/Interface
- Testar isoladamente
- Permitir customização sem permitir quebra (principle: open for extension)

O que é Middleware?

- Middleware é a “cola” do sistema moderno.
- É a camada que fica entre aplicações, serviços ou módulos, gerenciando comunicação, fluxo, segurança, infraestrutura e integrações transversais.
- Ele gera abstração para que os módulos não precisem saber detalhes internos uns dos outros.

Funções centrais do Middleware

- Comunicação
 - Padroniza como sistemas conversam:
 - mensagens,
 - eventos,
 - RPC,
 - REST.
- Interoperabilidade
 - Sistemas escritos em linguagens diferentes:
 - Java ↔ Python
 - Node.js ↔ .NET
 - Android ↔ Backend
 - O middleware cuida da tradução, protocolo e transporte.
- Integração
 - Unifica chamadas, filas e eventos.
 - Conecta microsserviços e sistemas legados.

Funções centrais do Middleware

- Orquestração
 - Define fluxos de negócio distribuídos.
 - Cuida de transações distribuídas (SAGA Pattern).
- Requisitos Não Funcionais (Cross-cutting)
 - Autenticação
 - Autorização (RBAC/ABAC)
 - Logging
 - Monitoramento
 - Rate limiting
 - Cache
 - Observabilidade
 - Tracing distribuído

Por que Middleware é crítico?

- Sem middleware, cada módulo teria que implementar:
 - protocolo de comunicação,
 - tratamento de erros,
 - segurança,
 - compatibilidade de formato,
 - autenticação,
 - serialização,
 - fila de mensagens...

- → O sistema ficaria impossível de manter.
 - O middleware centraliza tudo isso.

Tipos de Middleware

- Middleware de Comunicação
- Middleware de Integração
- Middleware de Suporte a Aplicações
- Middleware de Crosscutting

Middleware de Comunicação

- Cuida da troca de mensagens e protocolos.
- Exemplos:
 - REST (HTTP/JSON)
 - gRPC (RPC de alta performance)
 - WebSockets (tempo real)
 - Message Brokers
 - RabbitMQ
 - Kafka
 - ActiveMQ
 - SQS
- Usos típicos:
 - sistemas distribuídos,
 - comunicação entre microserviços,
 - chats em tempo real,
 - IoT,
 - streaming,
 - notificações push.

Middleware de Integração

- Suporta integrações complexas entre aplicações corporativas.
- Exemplos:
 - ESB – Enterprise Service Bus
 - Mule ESB, WSO2, Apache Camel
 - Service Mesh
 - Istio, Linkerd
 - Controle de tráfego, segurança mTLS, observabilidade
 - iPaaS
 - Microsoft Logic Apps, Zapier empresarial, AWS Step Functions
- Usos típicos:
 - Coordenar fluxos entre sistemas diferentes
 - Orquestrar serviços
 - Migrar legado para microsserviços

Middleware de Suporte a Aplicações

- Exemplos:
 - ORMs (Hibernate, SQLAlchemy, Prisma, TypeORM)
 - → abstraem banco de dados
 - Transaction Managers
 - → garantem ACID/SAGA
 - Identity Providers
 - → Keycloak, Auth0, Cognito
 - Schedulers
 - → Quartz, Celery, cron
- Usos:
 - Persistência
 - Transações
 - Autenticação/autorização
 - Escalonamento de tarefas

Middleware de Crosscutting

- Fornecem serviços transversais, usados em todo o sistema.
- Exemplos:
 - Logging (ELK, Loki)
 - Monitoramento (Prometheus, Grafana)
 - Telemetria OpenTelemetry
 - Caching (Redis, Memcached)
 - Rate limiting (Kong, Envoy)
 - Auditoria
 - Encryption middleware
- Benefícios:
 - Centraliza preocupações compartilhadas
 - Evita duplicação de código
 - Aumenta segurança e rastreabilidade

Boas práticas na criação de middleware

- Deve ser transparente ao restante da aplicação
- Não deve alterar a lógica principal
- Deve ser executado em pé da aplicação
- Não duplicar responsabilidades existentes
- Deve ser desabilitável ou substituível

Criar um componente reutilizável

- Construir um componente “CardUsuario”
- Objetivo: criar um componente reutilizável que recebe props e exibe conteúdo padronizado.
- Exemplo com Vue.js:

```
<template>
  <div class="card">
    
    <h3>{{ nome }}</h3>
    <p>{{ email }}</p>
  </div>
</template>

<script>
export default {
  name: "CardUsuario",
  props: {
    nome: String,
    email: String,
    foto: String
  }
}
</script>
```

Middleware: Logger de Requisições

- Este middleware irá registrar no console:
 - Método HTTP (GET, POST, etc.)
 - URL acessada
 - Horário da requisição
 - Tempo de resposta
- É um middleware real, igual aos usados em sistemas profissionais.

Middleware: Logger de Requisições

- Inicializar o projeto Node
 - npm init -y
- Instalar o Express
 - npm install express

```

aula-middleware/
├── server.js
├── middleware/
│   ├── requestId.js
│   ├── logger.js
│   ├── auditLogger.js
│   ├── slowRequest.js
│   └── errorHandler.js
├── utils/
│   └── sanitize.js
├── logs/
│   ├── access.log
│   ├── error.log
│   ├── audit.log
│   └── slow.log
└── package.json
  
```

Vantagens

- Observar (logging estruturado + request tracing)
- Auditar (LGPD / compliance / trilha de ações)
- Garantir segurança (fraude, invasão, payloads)
- Controlar performance (requisições lentas)
- Diagnosticar erros rapidamente
- Padronizar APIs em microserviços

Middleware de Autorização por Token

- Este middleware verifica se a requisição contém um header chamado X-API-KEY, e só permite o acesso se o token for válido.
- Se não for, ele bloqueia com HTTP 401.

Middleware de Autorização por Token

- Instale o FastAPI e Uvicorn:
 - `pip install fastapi uvicorn`

```
python-middleware/
├── main.py
└── middleware/
    └── api_key_middleware.py
```

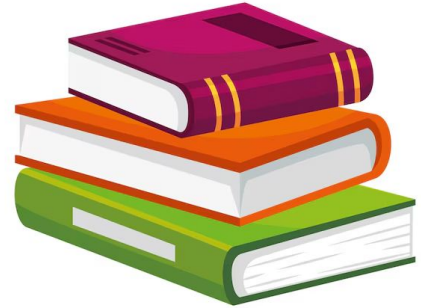
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

